K.R. MANGALAM UNIVERSITY

Database Management Lab File
(ENCS205)

Bachelor of Technology
in
Computer Science and Engineering

Submitted by
Krishna Vohra (2401010066)

B. Tech CSE (Core) – 4 Semester

Submitted To:
MS. Mansi Kajal

School of Engineering & Technology

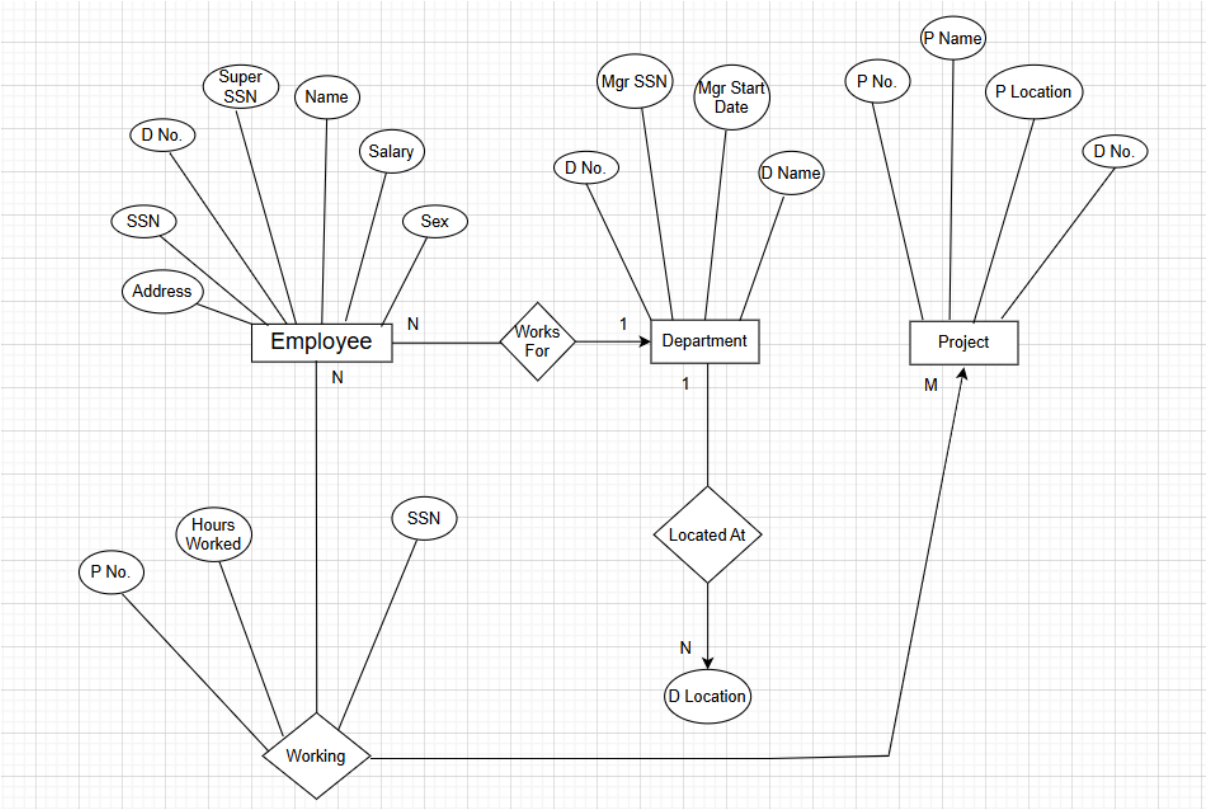
K. R. MANGALAM UNIVERSITY Sohna, Haryana
122103, India Jan-2026

Experiment 1

1. **Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:**

COMPANY DATABASE:

EMPLOYEE (SSN, Name, Address, Sex, Salary, Super SSN, D No.) DEPARTMENT (D No., D Name, Mgr SSN, Mgr Start Date) DLOCATION (D Location)
PROJECT (P No., P Name, P Location, D No.)
WORKING (SSN, P No., Hours Worked)



Explanation of ER Diagram – Company Database :

- This ER diagram represents a company database that stores information about employees, departments, locations, and projects.

- The Employee entity stores details such as SSN, name, address, sex, salary, supervisor SSN, and department number. Each employee works for a department. This relationship is shown using WORKS FOR. Many employees can work for one department, but each employee works for only one department.
- The Department entity stores department information such as

department number, department name, manager SSN, and manager start date. A department can have multiple locations. This is shown using the LOCATED AT relationship between Department and D Location.

- The Project entity stores information about projects such as project number, project name, project location, and department number. Each project is controlled by one department, and one department can control many projects.

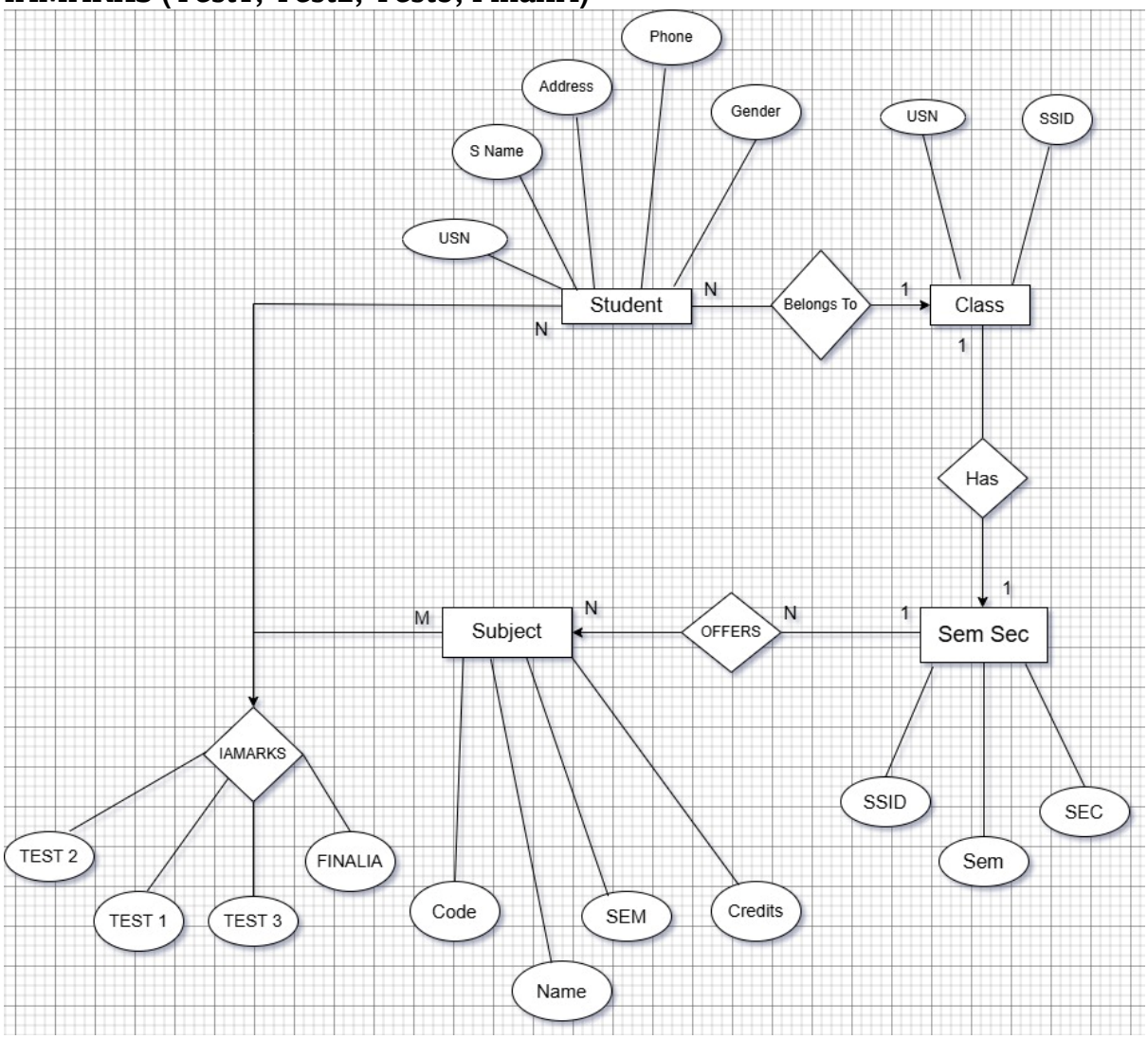
- Employees work on different projects. This many-to-many relationship is shown using the WORKS ON relationship. The number of hours an employee works on a project is stored as an attribute of this relationship.

EXPERIMENT-2

2. Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COLLEGE DATABASE:

STUDENT (USN, S Name, Address, Phone, Gender) SEMSEC (SSID, Sem, SEC)
CLASS (USN, SSID)
SUBJECT (Code, Name, Sem, Credits)
IAMARKS (Test1, Test2, Test3, FinalIA)



Explanation of ER Diagram – College Database :

- This ER diagram represents a college database that stores information about students, classes, semester–sections, subjects, and internal

assessment marks.

- In this diagram, the Student entity stores the basic details of a student such as USN, student name, address, phone number, and gender. Every student belongs to a particular class. This relationship is shown using the BELONGS TO relationship. Many students can belong to the same class, but a student can belong to only one class.

- The Class entity represents a group of students. Each class is associated with exactly one semester and section, which is represented by the SemSec entity. The relationship between Class and SemSec is one-to-one, meaning each class has only one semester-section and each semester-section belongs to one class.

- The SemSec entity stores information about the semester and section. A semester-section offers multiple subjects. This is shown by the OFFERS relationship between SemSec and Subject. One semester-section can offer many subjects, but each subject is offered in one specific semester-section.

- The Subject entity stores subject details such as subject code, subject name, semester, and credits. Students study multiple subjects, and each subject is studied by many students. This many-to-many relationship is represented using the IAMARKS relationship.

- The IAMARKS relationship stores the internal assessment marks of students for each subject. It includes Test1, Test2, Test3, and Final IA. This relationship connects Student and Subject and stores marks for every student–subject combination.

Experiment 3

Database Schema for a customer-sale scenario
Customer(Cust id : integer, cust_name: string)
Item(item_id: integer, item_name: string, price: integer)
Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)
For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.

```
CREATE DATABASE IF NOT EXISTS CustomerSaleDB;
USE CustomerSaleDB;

CREATE TABLE Customer (
   cust_id INT PRIMARY KEY,
   cust_name VARCHAR(100) NOT NULL
);


CREATE TABLE Item (
   item_id INT PRIMARY KEY,
   item_name VARCHAR(100) NOT NULL,
   price INT NOT NULL CHECK (price > 0)
);

CREATE TABLE Sale (
   bill_no INT,
   bill_date DATE NOT NULL,
   cust_id INT,
   item_id INT,
   qty_sold INT NOT NULL CHECK (qty_sold > 0),
   PRIMARY KEY (bill_no, item_id),
   FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
   FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

	cust_id	cust_name
*	NULL	NULL

	item_id	item_name	price
*	NULL	NULL	NULL

	bill_no	bill_date	cust_id	item_id	qty_sold
*	NULL	NULL	NULL	NULL	NULL

2. Insert around 10 records in each of the tables.

bill_no	cust_name	item_id
1001	Ishaan	101
1002	Mahin	102
1003	Aditya	104
1004	Ayush	105
1005	Priya	108
1006	Karan	110
1007	Neha	103
1008	Rohit	109
1009	Simran	106
1010	Vikas	107

```
INSERT INTO Customer VALUES
(1,'Ishaan'),
(2,'Mahin'),
(3,'Aditya'),
(4,'Ayush'),
(5,'Priya'),
(6,'Karan'),
(7,'Neha'),
(8,'Rohit'),
(9,'Simran'),
(10,'Vikas');

INSERT INTO Item VALUES
(101,'Keyboard',50000),
(102,'Mouse',500),
(103,'Keyboard',800),
(104,'Monitor',12000),
(105,'USB Cable',200),
(106,'Headphones',1500),
(107,'Charger',900),
(108,'Tablet',25000),
(109,'Smartphone',30000),
(110,'Pen Drive',700);

INSERT INTO Sale VALUES
(1001,CURDATE(),1,101,1),
(1002,CURDATE(),2,102,2),
(1003,CURDATE(),3,104,1),
(1004,CURDATE(),4,105,3),
(1005,CURDATE(),5,108,1),
(1006,CURDATE(),6,110,4),
(1007,CURDATE(),7,103,2),
(1008,CURDATE(),8,109,1),
(1009,CURDATE(),9,106,1),
(1010,CURDATE(),10,107,2);
```

cust_id	cust_name
1	Ishaan
2	Mahin
3	Aditya
4	Ayush
5	Priya
6	Karan
7	Neha
8	Rohit
9	Simran
10	Vikas
NULL	NULL

item_id	item_name	price
101	Keyboard	50000
102	Mouse	500
103	Keyboard	800
104	Monitor	12000
105	USB Cable	200
106	Headphones	1500
107	Charger	900
108	Tablet	25000
109	Smartphone	30000
110	Pen Drive	700
NULL	NULL	NULL

3. List all the bills for the current date with the customer names and item numbers.

```
SELECT s.bill_no, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();
```

bill_no	bill_date	cust_id	item_id	qty_sold
1001	2026-02-28	1	101	1
1002	2026-02-28	2	102	2
1003	2026-02-28	3	104	1
1004	2026-02-28	4	105	3
1005	2026-02-28	5	108	1
1006	2026-02-28	6	110	4
1007	2026-02-28	7	103	2
1008	2026-02-28	8	109	1
1009	2026-02-28	9	106	1
1010	2026-02-28	10	107	2
NULL	NULL	NULL	NULL	NULL

4. List the total Bill details with the quantity sold, price of the item and the final amount.

```
SELECT
   s.bill_no,
   s.qty_sold,
   i.price,
   (s.qty_sold * i.price) AS final_amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

bill_no	qty_sold	price	final_amount
1001	1	50000	50000
1002	2	500	1000
1003	1	12000	12000
1004	3	200	600
1005	1	25000	25000
1006	4	700	2800
1007	2	800	1600
1008	1	30000	30000
1009	1	1500	1500
1010	2	900	1800

5. List the details of the customer who have bought a product which has a price>200.

```
SELECT DISTINCT c.cust_id, c.cust_name
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

cust_id	cust_name
1	Ishaan
2	Mahin
7	Neha
3	Aditya
9	Simran
10	Vikas
5	Priya
8	Rohit
6	Karan

6. Give a count of how many products have been bought by each customer

```
SELECT
   c.cust_id,
   c.cust_name,
   SUM(s.qty_sold) AS total_products_bought
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_id, c.cust_name;
```

	cust_id	cust_name	total_products_bought
▶	1	Ishaan	1
	2	Mahin	2
	3	Aditya	1
	4	Ayush	3
	5	Priya	1
	6	Karan	4
	7	Neha	2
	8	Rohit	1
	9	Simran	1
	10	Vikas	2

7. Give a list of products bought by a customer having cust_id as 5.

```
SELECT i.item_id, i.item_name
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;
```

	item_id	item_name
▶	108	Tablet

8. List the item details which are sold as of today.

```
SELECT DISTINCT i.*
FROM Item i
JOIN Sale s ON i.item_id = s.item_id
WHERE s.bill_date = CURDATE();
```

	item_id	item_name	price
▶	101	Keyboard	50000
	102	Mouse	500
	104	Monitor	12000
	105	USB Cable	200
	108	Tablet	25000
	110	Pen Drive	700
	103	Keyboard	800
	109	Smartphone	30000
	106	Headphones	1500
	107	Charger	900

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

```
CREATE OR REPLACE VIEW Bill_View AS
SELECT
    s.bill_no,
    s.bill_date,
```

```
    s.cust_id,
    s.item_id,
    i.price,
    s.qty_sold,
    (s.qty_sold * i.price) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
SELECT * FROM Bill_View;
```

bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
1001	2026-02-28	1	101	50000	1	50000
1002	2026-02-28	2	102	500	2	1000
1003	2026-02-28	3	104	12000	1	12000
1004	2026-02-28	4	105	200	3	600
1005	2026-02-28	5	108	25000	1	25000
1006	2026-02-28	6	110	700	4	2800
1007	2026-02-28	7	103	800	2	1600
1008	2026-02-28	8	109	30000	1	30000
1009	2026-02-28	9	106	1500	1	1500
1010	2026-02-28	10	107	900	2	1800

10. Create a view which lists the daily sales date wise for the last one week

```
CREATE OR REPLACE VIEW Weekly_Sales AS
SELECT
    s.bill_date,
    SUM(s.qty_sold * i.price) AS total_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY s.bill_date;
SELECT * FROM Weekly_Sales;
```

Bill_date:	2026-02-28
Total_sales:	126300

K.R. MANGALAM UNIVERSITY

Database Management Lab File (ENCS205)

Bachelor of Technology
in
Computer Science and Engineering

Submitted by
Krishna Vohra (2401010066)

B. Tech CSE (Core) – 4 Semester

Submitted To:
MS. Mansi Kajal

School of Engineering & Technology

K. R. MANGALAM UNIVERSITY Sohna, Haryana 122103, India Jan-2026

QUESTION : Assignment 4

Consider the following table: Table: CLASS

ID	Name
1	Bravo
2	Alex
4	Cheng

Give the output of the following SQL script:

- **INSERT INTO class VALUES (5,’Rahul’);**
- **COMMIT;**
- **UPDATE class SET name = ‘Abhijeet’ WHERE id= ‘5’;**
- **SAVEPOINT A;**
- **INSERT INTO class VALUES (6, ‘Chris’);**
- **SAVEPOINT B;**
- **INSERT INTO class VALUES (7, ‘Bravo’);**
- **SAVEPOINT C**
- **SELECT * FROM class;**
- **ROLLBACK TO B;**
- **SELECT * FROM class;**
- **ROLLBACK TO A;**

CODE :

```
SET autocommit = 0;


CREATE TABLE class (
   id INT,
   name VARCHAR(50)
);



INSERT INTO class VALUES (1,'Bravo');
INSERT INTO class VALUES (2,'Alex');
```

```
INSERT INTO class VALUES (4,'Cheng');

INSERT INTO class VALUES (5,'Rahul');
COMMIT;

UPDATE class SET name='Abhijeet' WHERE id='5';

SAVEPOINT A;

INSERT INTO class VALUES (6,'Chris');

SAVEPOINT B;

INSERT INTO class VALUES (7,'Bravo');

SAVEPOINT C;

SELECT * FROM class;

ROLLBACK TO B;

SELECT * FROM class;

ROLLBACK TO A;
```

Table :

Result Grid

id	name
1	Bravo
2	Alex
4	Cheng
5	Abhijeet
6	Chris
NULL	NULL

Assignment (4.2)

Consider the following table: Table: ACCOUNT_TXN

Acc_ID	Holder_Name	Balance
101	Aman	5000
102	Bhavna	7000
103	Chirag	6000

Perform the following TCL-based SQL script and write the output after each SELECT statement:

CODE :

```
SET autocommit = 0;

DROP TABLE IF EXISTS ACCOUNT_TXN;

CREATE TABLE ACCOUNT_TXN (
Acc_ID INT PRIMARY KEY,
   Holder_Name VARCHAR(50),
   Balance INT
);

INSERT INTO ACCOUNT_TXN VALUES
(101,'Aman',5000),
(102,'Deepak',7000),
(103,'Chirag',6000);

SELECT * FROM ACCOUNT_TXN;

START TRANSACTION;

INSERT INTO ACCOUNT_TXN VALUES (104,'Diya',4000);

SAVEPOINT S1;
```

```
UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101;

UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102;

SAVEPOINT S2;

DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103;

INSERT INTO ACCOUNT_TXN VALUES (105,'Eshan',3000);

SAVEPOINT S3;

UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID = 102;

SELECT * FROM ACCOUNT_TXN;

ROLLBACK TO S2;

SELECT * FROM ACCOUNT_TXN;

INSERT INTO ACCOUNT_TXN VALUES (106,'Farah',4500);

SAVEPOINT S4;

UPDATE ACCOUNT_TXN SET Balance = Balance + 500
WHERE Acc_ID IN (101,104);

SELECT * FROM ACCOUNT_TXN;

ROLLBACK TO S4;

SELECT * FROM ACCOUNT_TXN;

COMMIT;
```

OUTPUT :

Result Grid | Filter Rows:

Acc_ID	Holder_Name	Balance
101	Aman	6500
102	Deepak	6000
103	Chirag	6000
104	Diya	4000
106	Farah	4500
NULL	NULL	NULL

DBMS Experiment 5

EMPLOYEES (Employee_Id, First_Name, Last_Name, Email, Phone_Number, Hire_Date, Job_Id, Salary, Commission_Act, Manager_Id, Department_Id) CREATE database emp;

```
use emp;

CREATE TABLE EMPLOYEES (
   Employee_Id INT PRIMARY KEY,
   First_Name VARCHAR(50),
   Last_Name VARCHAR(50),
   Email VARCHAR(100),
   Phone_Number VARCHAR(20),
   Hire_Date DATE,
   Job_Id VARCHAR(20),
   Salary INT,
   Commission_Act DECIMAL(5,2),
   Manager_Id INT,
   Department_Id INT
);
INSERT INTO EMPLOYEES VALUES
(101, 'John', 'Smith', 'john.smith@email.com', '9876543210', '2020-01-15', 'IT_PROG', 6000, NULL, 100, 60),
(102, 'Sara', 'Connor', 'sara.connor@email.com', '9876543211', '2019-03-10', 'HR_REP', 4500, NULL, 101, 70),
(103, 'Mike', 'Austin', 'mike.austin@email.com', '9876543212', '2021-06-20', 'SA_REP', 5000, 0.10, 100, 80),
(104, 'Emma', 'Watson', 'emma.watson@email.com', '9876543213', '2018-11-05', 'IT_PROG', 7000, NULL, 101, 60),
(105, 'Liam', 'Brown', 'liam.brown@email.com', '9876543214', '2022-02-18', 'SA_REP', 4800, 0.15, 102, 80),
(106, 'Olivia', 'Davis', 'olivia.davis@email.com', '9876543215', '2020-07-25', 'HR_REP', 4200, NULL, 101, 70),
```

(107, 'Noah', 'Wilson', 'noah.wilson@email.com', '9876543216', '2017-09-30', 'IT_PROG', 8000, NULL, 100, 60),

(108, 'Ava', 'Taylor', 'ava.taylor@email.com', '9876543217', '2019-12-11', 'SA_REP', 4700, 0.12, 102, 80),

(109, 'Ethan', 'Anderson', 'ethan.anderson@email.com', '9876543218', '2021-04-14', 'IT_PROG', 5200, NULL, 101, 60),

(110, 'Sophia', 'Thomas', 'sophia.thomas@email.com', '9876543219', '2018-08-19', 'HR_REP', 4600, NULL, 100, 70),

(111, 'James', 'Jackson', 'james.jackson@email.com', '9876543220', '2020-05-23', 'SA_REP', 4900, 0.20, 102, 80),

(112, 'Isabella', 'White', 'isabella.white@email.com', '9876543221', '2019-01-30', 'IT_PROG', 6100, NULL, 101, 60),

(113, 'Lucas', 'Harris', 'lucas.harris@email.com', '9876543222', '2022-03-12', 'HR_REP', 4300, NULL, 100, 70),

(114, 'Mia', 'Martin', 'mia.martin@email.com', '9876543223', '2021-10-07', 'SA_REP', 5100, 0.18, 102, 80),

(115, 'William', 'Austin', 'william.austin@email.com', '9876543224', '2020-12-01', 'IT_PROG', 7500, NULL, 100, 60);

(a) Find out the employee id, names, salaries of all the employees

SELECT Employee_Id, First_Name, Last_Name, Salary

FROM EMPLOYEES;

Result Grid | Filter Rows: | Edit: | Export/Import:

Employee_Id	First_Name	Last_Name	Salary
101	John	Smith	6000
102	Sara	Connor	4500
103	Mike	Austin	5000
104	Emma	Watson	7000
105	Liam	Brown	4800
106	Olivia	Davis	4200
107	Noah	Wilson	8000
108	Ava	Taylor	4700
109	Ethan	Anderson	5200
110	Sophia	Thomas	4600
111	James	Jackson	4900
112	Isabella	White	6100
113	Lucas	Harris	4300
114	Mia	Martin	5100
115	William	Austin	7500
NULL	NULL	NULL	NULL

EMPLOYEES 1 × | EMPLOYEES 2 | EMPLOYEES 3 | EMPLOYEES 4 | EMPLOYEES 5 | EMPL

(b) List out the employees who works under manager 100

SELECT *

FROM EMPLOYEES

WHERE Manager_Id = 100;

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Employee_Id	First_Name	Last_Name	Email	Phone_Number	Hire_Date	Job_Id	Salary	Commission_Act	Manager_Id	Department_Id
101	John	Smith	john.smith@email.com	9876543210	2020-01-15	IT_PROG	6000	NULL	100	60
103	Mike	Austin	mike.austin@email.com	9876543212	2021-06-20	SA_REP	5000	0.10	100	80
107	Noah	Wilson	noah.wilson@email.com	9876543216	2017-09-30	IT_PROG	8000	NULL	100	60
110	Sophia	Thomas	sophia.thomas@email.com	9876543219	2018-08-19	HR_REP	4600	NULL	100	70
113	Lucas	Harris	lucas.harris@email.com	9876543222	2022-03-12	HR_REP	4300	NULL	100	70
115	William	Austin	william.austin@email.com	9876543224	2020-12-01	IT_PROG	7500	NULL	100	60
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

EMPLOYEES 1 | EMPLOYEES 2 | EMPLOYEES 3 | EMPLOYEES 4 | EMPLOYEES 5 | EMPLOYEES 6 | Apply

(c) Find the names of the employees who have a salary greater than or equal to 4800

SELECT First_Name, Last_Name

FROM EMPLOYEES

WHERE Salary >= 4800;

Result Grid | Filter Rows: | Export: | Wrap C

First_Name	Last_Name
John	Smith
Mike	Austin
Emma	Watson
Liam	Brown
Noah	Wilson
Ethan	Anderson
James	Jackson
Isabella	White
Mia	Martin
William	Austin

EMPLOYEES 1 | EMPLOYEES 2 | EMPLOYEES 3 | EMPLOYEES 4

(d) List out the employees whose last name is 'AUSTIN'

SELECT *

FROM EMPLOYEES

WHERE Last_Name = 'AUSTIN';

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Employee_Id	First_Name	Last_Name	Email	Phone_Number	Hire_Date	Job_Id	Salary	Commission_Act	Manager_Id	Department_Id
103	Mike	Austin	mike.austin@email.com	9876543212	2021-06-20	SA_REP	5000	0.10	100	80
115	William	Austin	william.austin@email.com	9876543224	2020-12-01	IT_PROG	7500	NULL	100	60
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

EMPLOYEES 1 | EMPLOYEES 2 | EMPLOYEES 3 | EMPLOYEES 4 | EMPLOYEES 5 | EMPLOYEES 6 | Apply

(e) Find the names of the employees who works in departments 60,70 and 80

SELECT First_Name, Last_Name

FROM EMPLOYEES

WHERE Department_Id IN (60, 70, 80);

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

First_Name	Last_Name
John	Smith
Sara	Connor
Mike	Austin
Emma	Watson
Liam	Brown
Olivia	Davis
Noah	Wilson
Ava	Taylor
Ethan	Anderson
Sophia	Thomas
James	Jackson
Isabella	White
Lucas	Harris
Mia	Martin
William	Austin

EMPLOYEES 1 | EMPLOYEES 2 | EMPLOYEES 3 | EMPLOYEES 4 | EMPLOYEES 5

(f) Display the unique Manager_Id

SELECT DISTINCT Manager_Id

FROM EMPLOYEES;

Result Grid | Filter Rows:

Manager_Id
100
101
102

DBMS Experiment 6

(Exercise on updating records in table) Create Client_master with the following fields (ClientNO, Name, Address, City, State, bal_due); Create database Client_master; use Client_master;

```
CREATE TABLE Client_master (
   ClientNO VARCHAR(10) PRIMARY KEY,
   Name VARCHAR(50),
   Address VARCHAR(100),
   City VARCHAR(50),     State
VARCHAR(50),     bal_due
DECIMAL(10,2)
);
```

(a) Insert five records

```
INSERT INTO Client_master VALUES
('C106', 'Anjali Kapoor', 'Sector 18', 'Noida', 'Uttar Pradesh', 4800),
('C107', 'Vikas Malhotra', 'Connaught Place', 'Delhi', 'Delhi', 8000),
('C108', 'Sneha Reddy', 'Banjara Hills', 'Hyderabad', 'Telangana', 6200),
('C109', 'Arjun Nair', 'Marine Drive', 'Mumbai', 'Maharashtra', 5300),
('C110', 'Karan Patel', 'SG Highway', 'Ahmedabad', 'Gujarat', 4500);
```

Result Grid | Filter Rows: | Edit: | Export/Import:

ClientNO	Name	Address	City	State	bal_due
C106	Anjali Kapoor	Sector 18	Noida	Uttar Pradesh	4800.00
C107	Vikas Malhotra	Connaught Place	Delhi	Delhi	8000.00
C108	Sneha Reddy	Banjara Hills	Hyderabad	Telangana	6200.00
C109	Arjun Nair	Marine Drive	Mumbai	Maharashtra	5300.00
C110	Karan Patel	SG Highway	Ahmedabad	Gujarat	4500.00
NULL	NULL	NULL	NULL	NULL	NULL

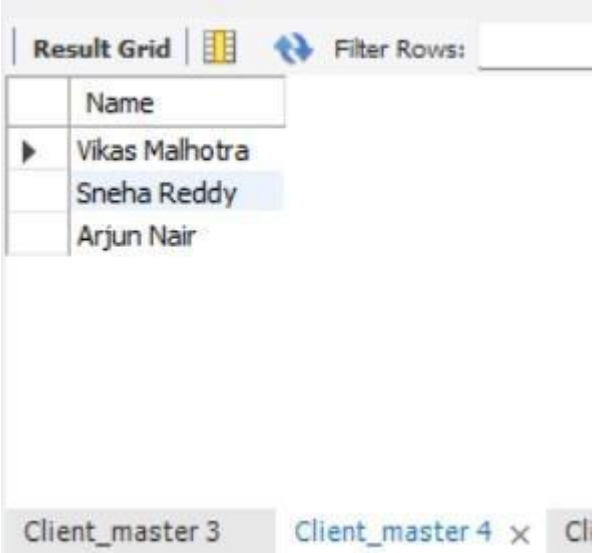
Client_master 3 × | Client_master 4 | Client_master 5 | Client12 6

(b) Find the names of clients whose bal_due> 5000 .

SELECT Name

FROM Client_master

WHERE bal_due > 5000;



(c) Change the bal_due of ClientNO “ C123” to Rs. 5100

UPDATE Client_master

SET bal_due = 5100

WHERE ClientNO = 'C123';

Result Grid | Filter Rows: | Edit: | Export/Import:

ClientNO	Name	Address	City	State	bal_due
C106	Anjali Kapoor	Sector 18	Noida	Uttar Pradesh	4800.00
C107	Vikas Malhotra	Connaught Place	Delhi	Delhi	8000.00
C108	Sneha Reddy	Banjara Hills	Hyderabad	Telangana	6200.00
C109	Arjun Nair	Marine Drive	Mumbai	Maharashtra	5300.00
C110	Karan Patel	SG Highway	Ahmedabad	Gujarat	4500.00
NULL	NULL	NULL	NULL	NULL	NULL

Client_master 3 | Client_master 4 | Client_master 5 × | Client12 6

(d) Change the name of Client_master to Client12.

ALTER TABLE Client_master

RENAME TO Client12;

(e) Display the bal_due heading as "BALANCE"

SELECT ClientNO, Name, Address, City, State, bal_due AS BALANCE FROM Client12;

Result Grid | Filter Rows: | Edit: | Export/Import:

ClientNO	Name	Address	City	State	BALANCE
C106	Anjali Kapoor	Sector 18	Noida	Uttar Pradesh	4800.00
C107	Vikas Malhotra	Connaught Place	Delhi	Delhi	8000.00
C108	Sneha Reddy	Banjara Hills	Hyderabad	Telangana	6200.00
C109	Arjun Nair	Marine Drive	Mumbai	Maharashtra	5300.00
C110	Karan Patel	SG Highway	Ahmedabad	Gujarat	4500.00
NULL	NULL	NULL	NULL	NULL	NULL

Client_master 3 | Client_master 4 | Client_master 5 | Client12 6 ×

Experiment 7

Krishna Vohra – 2401010066

Objective: Rollback and Commit commands Create Teacher table with the following fields (Name, DeptNo, Date of joining, DeptName, Location, Salary)

a) Insert five records.
b) Give Increment of 25% salary for Mathematics Department.
c) Perform Rollback command.
d) Give increment of 15% salary for Commerce Department.
e) Perform Commit command.

Code:

```
CREATE TABLE Teacher (
  Name VARCHAR(50),
  DeptNo INT,
  Date_of_Joining DATE,
  DeptName VARCHAR(50),
  Location VARCHAR(50),
  Salary NUMBER(10,2)
```

);

a) Insert 5 records

INSERT INTO Teacher (TeacherName, DeptNo, DateOfJoining, DeptName, LocationName, SalaryAmt)

VALUES

('amit kumar', 10, '2021-01-10', 'mathematics', 'delhi', 40000),

('sunita devi', 20, '2020-03-15', 'commerce', 'mumbai', 35000),

('rahul sharma', 10, '2022-05-22', 'mathematics', 'delhi', 42000),

('priya nair', 30, '2019-11-01', 'science', 'kochi', 50000),

('aniket das', 20, '2023-02-14', 'commerce', 'kolkata', 32000);

b) Increase salary by 25% for Mathematics

UPDATE Teacher
SET Salary = Salary + (Salary * 0.25)
WHERE DeptName = 'Mathematics';

	TeacherName	DeptNo	DateOfJoining	DeptName	LocationName	SalaryAmt
▶	amit kumar	10	2021-01-10	mathematics	delhi	50000
	rahul sharma	10	2022-05-22	mathematics	delhi	52500

(c) Perform Rollback

ROLLBACK;

	TeacherName	DeptNo	DateOfJoining	DeptName	LocationName	SalaryAmt
▶	amit kumar	10	2021-01-10	mathematics	delhi	40000
	rahul sharma	10	2022-05-22	mathematics	delhi	42000

(d) Increase salary by 15% for Commerce

```
UPDATE Teacher
SET Salary = Salary + (Salary * 0.15)
WHERE DeptName = 'Commerce';
```

(e) Commit changes

```
COMMIT;
```

	TeacherName	DeptNo	DateOfJoining	DeptName	LocationName	SalaryAmt
▶	amit kumar	10	2021-01-10	mathematics	delhi	40000
	sunita devi	20	2020-03-15	commerce	mumbai	40250
	rahul sharma	10	2022-05-22	mathematics	delhi	42000
	priya nair	30	2019-11-01	science	kochi	50000
	aniket das	20	2023-02-14	commerce	kolkata	36800

Experiment 8

Krishna – 2401010066

AIM: Create Sales table with the following fields (Sales No, Salesname, Branch, Salesamount, DOB)

a) Insert five records.
b) Calculate total salesamount in each branch.
c) Calculate average salesamount in each branch.
d) Display all the salesmen, DOB who are born in the month of December as day in character format i.e. 21-Dec-09.
e) Display the name and DOB of salesman in alphabetical order of the month.

Create Table

```
CREATE TABLE Sales (
   Sales_No INT,
   Salesname VARCHAR(50),
   Branch VARCHAR(50),
   Salesamount NUMBER(10,2),
   DOB DATE
);
```

a) Insert 5 records

INSERT INTO Sales (SalesNo, SalesName, BranchName, SalesAmount, DateOfBirth)

VALUES

(101, 'arnav gupta', 'north', 45000, '1995-12-21'),

(102, 'sana sheikh', 'south', 32000, '1998-05-15'),

(103, 'kabir malhotra', 'north', 55000, '1996-12-09'),

(104, 'isha patel', 'east', 40000, '1997-12-05'),

(105, 'vihaan nair', 'south', 28000, '1999-08-22');

b) Total sales amount in each branch

SELECT Branch, SUM(Salesamount) AS Total_Sales
FROM Sales
GROUP BY Branch;

	BranchName	TotalSales
▶	north	100000
	south	60000
	east	40000

c) Average sales amount in each branch

SELECT Branch, AVG(Salesamount) AS Avg_Sales
FROM Sales
GROUP BY Branch;

	BranchName	AverageSales
▶	north	50000.0000
	south	30000.0000
	east	40000.0000

d) Salesmen born in December (formatted DOB)

```
SELECT Salesname,
    TO_CHAR(DOB, 'DD-MON-YY') AS DOB
FROM Sales
WHERE TO_CHAR(DOB, 'MM') = '12';
```

	SalesName	FormattedDOB
▶	arnav gupta	21-Dec-95
	kabir malhotra	09-Dec-96
	isha patel	05-Dec-97

e) Order by month alphabetically

```
SELECT Salesname, DOB
FROM Sales
ORDER BY TO_CHAR(DOB, 'Month');
```

	SalesName	DateOfBirth
▶	vihaan nair	1999-08-22
	arnav gupta	1995-12-21
	kabir malhotra	1996-12-09
	isha patel	1997-12-05
	sana sheikh	1998-05-15